ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА РАЗРАБОТКИ ПО · ЛАБОРАТОРНАЯ РАБОТА

Полный цикл работы над фичей

Транспорт и формат · ручное тестирование · отладка · дефекты и релиз — одна работа, весь курс

Вы — junior-инженер команды. Система «ИТ-снабжение» (заявка на оснащение рабочего места) собрана, релиз — завтра. Работа проходит четыре этапа: вы разбираете обмены данных системы, тестируете её вручную, расследуете сбой по материалам и оформляете итог в виде дефект-репортов и решения о релизе. Всё выполняется самостоятельно; материалы — в папке `materials`, сама система — `app/index.html` (открывается двойным кликом, интернет не нужен).

Что я хочу видеть в работе

1. **Ответ с доказательством.** На каждый вопрос — ответ и строка-цитата из материала (или шаги в приложении).
2. **Цепочка рассуждения.** Не «сломалась база», а: заголовок говорит X → строка Y → значит причина Z.
3. **План раньше действий.** В части 2 чек-лист составляется до тестирования.
4. **Ссылка на эталон.** «Ожидание» в репортах опирается на пункт требований.
5. **Обоснованные решения.** Severity, триаж и вердикт по релизу — с аргументами, а не «на глаз».

01 Транспорт и формат данных

25 мин

1.1 · Чтение HTTP-обменов — `materials/http-dump.txt`

Три записанных обмена системы с сервером (строки запроса отмечены «>», ответа — «<»). Ответьте:

1. Обмен A: почему запросов два, а не один? Какую роль сыграл заголовок **Location**? Какой итоговый код ответа увидит браузер?
2. Обмен A: что сервер сделал с заголовком **Set-Cookie** и зачем система будет отправлять его обратно?
3. Обмен B: почему созданный заказ вернул код **201**, а не 200? Что лежало в теле запроса и что — в теле ответа?
4. Обмен C: в чём разница между кодами **404** и **500** — кто виноват в каждом случае и стоит ли повторять запрос?

1.2 · Сломанный JSON — `materials/order-broken.json`

Заявка в JSON содержит **три синтаксические ошибки** — документ не разбирается парсером.

1. Найдите и выпишите все три ошибки (с номерами строк).
2. Запишите исправленный документ целиком.
3. Какой заголовок должен сопровождать этот документ при отправке по HTTP?

1.3 · JSON против XML — `materials/order.xml`

Та же заявка, записанная в XML.

1. Посчитайте размер обеих версий в символах и запишите оба числа.
2. Какие два свойства XML делают его больше JSON на тех же данных?
3. Что XML умеет, а JSON — нет (одна вещь достаточно)?

02 Ручное тестирование системы

25 мин

Требования к системе (эталон для проверки)

- **Каталог и заявка.** Количество каждой позиции — целое **от 0 и выше**; кнопка «−» не должна уводить количество в минус.
- **Сумма.** Итог пересчитывается при **любом** изменении заявки: добавление, уменьшение, удаление, очистка.
- **Промокод IT10** — скидка **10% на заказ, применяется один раз**. Сумма в сообщении об успехе совпадает с суммой на экране.
- **Доставка** — 200 ₽; при сумме оборудования **от 5000 ₽ — бесплатно**.
- **Форма:** ФИО — не пустое; email — корректный формат **обязательно**; телефон — только цифры, 11 символов; дата выхода — не раньше следующего дня.
- **Заявку можно подать только с непустым списком.** После отправки: сообщение, заявка очищена, форма готова к новой.
- Тексты интерфейса — без опечаток.

2.1 · Чек-лист, затем прогон

Составьте чек-лист **минимум 12 пунктов** (области: количество, суммы, промокод, доставка, поля формы, отправка, тексты), затем выполните прогон: каждому пункту статус **passed / failed / blocked**. Дефектов в системе запас — **не меньше десяти**. Приёмы: границы (ровно 5000, 0, −1), пустые поля, неверный тип данных, повторные действия, сверка «экран ↔ сообщение об успехе».

Результат: чек-лист со статусами. Для себя отмечайте шаги — они понадобятся в части 4.

03 Отладка по материалам

15 мин

3.1 · След исключения — `materials/trace.txt`

1. Что за исключение и что именно оказалось null (текст следа отвечает буквально)?
2. Какая строка следа — «своя» для команды системы, и почему остальные — «чужие»?
3. Опишите первопричину своими словами: что разработчик не учёл?
4. Предложите исправление: где в коде должна быть проверка и что она делает?

3.2 · Утренний сбой — `materials/incident.log`

1. Выпишите таймлайн каскада: не меньше четырёх событий с временем.
2. Какой момент — первый аномальный, и почему начало не там, где ERROR?
3. Что произошло после перезапуска в 08:52 и что это говорит о первопричине?
4. Сформулируйте первопричину и предложите, что исправить (одно действие — минимум).

04 Дефекты, триаж, релиз

25 мин

4.1 · Дефект-репорты — минимум 4

ПОЛЕ	ЧТО ПИСАТЬ
Заголовок	краткий и точный: «Кнопка "−" уводит количество в минус»
Шаги	нумерованный список — воспроизводится с нуля любым человеком
Ожидание	по пункту требований — цитата или точный пересказ
Факт	что произошло — наблюдение, без оценок
Severity	critical / major / minor / trivial + обоснование одной строкой

Шкала: critical — пользоваться нельзя; major — важная функция неверна; minor — обход есть; trivial — косметика.

4.2 · Триаж

Разработчик успевает исправить только **три** дефекта до релиза. Какие три — и почему; какие два можно отложить — и почему допустимо.

4.3 · Решение о релизе

Сведите итоги: pass rate прогона (passed из проверенных), открытые дефекты по severity, вердикт **выпускаем / не выпускаем** и обоснование в 2–3 предложения.

Ориентир из практики: при открытых critical или провале ключевых сценариев релиз не идёт.

Формат сдачи

Один файл **otchet.txt**: ответы частей 1 и 3 (вопрос → ответ → цитата-доказательство), чек-лист части 2, репорты, триаж и решение части 4.

Работа рассчитана на пару; не успели — доработайте к следующему занятию.